

NexusLB: A Custom Reverse Proxy & Load Balancer

Project Proposal for UCS503P
Submitted to: Jhonsy Bansal
Thapar Institute of Engineering and Technology

Navjot Singh, CSED* Shaina Gera, CSED[†] Prabhgun Kaur, CSED[‡]

August 16, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the reliability and efficiency of distributed web services by ensuring intelligent traffic management, fault tolerance, and system visibility. While the broader goal is to understand and address challenges in modern server infrastructure, the immediate engineering objective is to build a measurable, health-aware reverse proxy and load balancer that can intelligently distribute requests, handle failures, and provide real-time operational insights.

2 Time-to-value

... primary heuristic

- We will deliver functional increments quickly by first implementing the core reverse proxy and request forwarding workflow, followed by routing strategies, health checks, and reliability features in short development iterations.
- We will prioritize a working and measurable system over feature completeness by validating each module independently and gradually integrating advanced capabilities such as failover, metrics, and dashboard visualization.
- Success will be defined by the ability to demonstrate real improvements in request distribution, failure handling, and system observability through controlled experiments and benchmark results.

3 Problem Statement

Modern web applications often experience unpredictable traffic patterns and require multiple backend servers to maintain availability, performance, and reliability. However, simply deploying multiple servers does not solve the problem of managing incoming requests effectively. A system is required to intelligently distribute traffic, detect failures, and provide visibility into the behaviour of backend services. Common challenges include:

- **Uneven traffic distribution:** Requests may be routed inefficiently, causing some backend servers to become overloaded while others remain underutilized.

*Roll No: 1024030313, Email: nsingh2_be24thapar.edu

[†]Roll No: 1024030316, Email: sgera_be24thapar.edu

[‡]Roll No: 1024030320, Email: pkaur1_be24thapar.edu

- **Lack of fault awareness:** Without continuous health monitoring, failed or unresponsive servers may continue receiving requests, leading to service disruptions and degraded user experience.
- **Limited system visibility:** Developers and administrators often lack a centralized view of request distribution, backend health, latency, throughput, and failure patterns.
- **Poor failure handling:** A backend failure during request processing can result in failed user requests unless mechanisms such as retries, failover, and timeout management are implemented.

Why this matters now (contextual relevance): As applications scale and depend on distributed backend infrastructure, efficient traffic management becomes critical for maintaining availability and performance. Building a health-aware reverse proxy and load balancer provides practical insight into how modern infrastructure systems handle reliability, scalability, and observability challenges.

4 Proposed Solution

... software proposition

4.1 Overview

The proposed solution is to build **NexusLB**, a custom reverse proxy and load balancer designed to intelligently manage traffic between clients and multiple backend servers. The system will act as a single entry point for incoming requests and will make routing decisions based on backend availability, load conditions, and configured balancing strategies.

Unlike basic load balancer implementations that only distribute requests using a fixed algorithm, NexusLB will focus on building a more realistic infrastructure component by combining traffic distribution with health monitoring, fault handling, and system observability. The system will consist of the following major components:

- **Reverse Proxy:** Acts as the communication layer between clients and backend servers. It receives incoming HTTP requests, forwards them to selected backend servers, and returns the generated responses back to clients.
- **Load Balancing Engine:** Responsible for selecting the appropriate backend server using multiple routing strategies such as Round Robin, Least Connections, and IP Hash. The strategy will be configurable based on system requirements.
- **Backend Registry:** Maintains information about available backend servers, including their addresses, current status, and connection details.
- **Health Checker:** Periodically monitors backend servers by sending health-check requests. Failed servers are automatically removed from the routing pool and restored once they become healthy again.
- **Reliability Layer:** Handles failures using request timeouts, controlled retries, and failover mechanisms to ensure that temporary backend failures do not cause complete service disruption.
- **Rate Limiter:** Controls incoming traffic by restricting excessive requests from clients and preventing backend servers from being overwhelmed during sudden traffic spikes.
- **Metrics and Monitoring System:** Collects operational information such as request count, latency, error rate, backend health, and traffic distribution.
- **Dashboard and Logging System:** Provides a real-time interface to visualize system behaviour and maintains structured logs for request tracing and debugging.

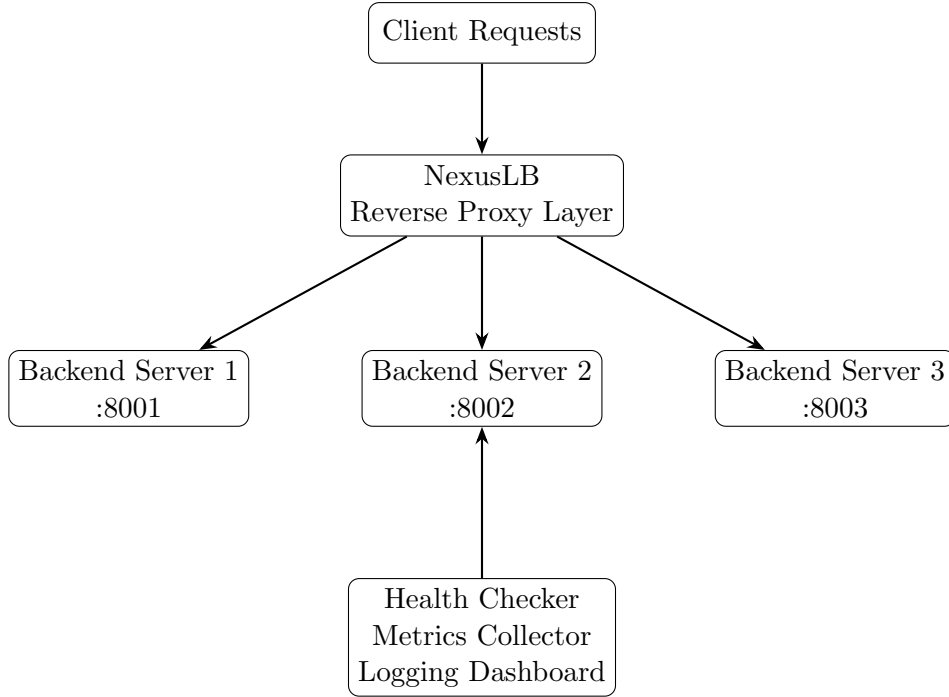


Figure 1: High-level architecture of NexusLB showing client interaction, reverse proxy layer, backend servers, and monitoring components.

4.2 Core Workflow

The core workflow of NexusLB follows a complete request lifecycle from receiving a client request to updating system metrics:

1. A client sends an HTTP request to NexusLB, which acts as the single entry point for all incoming traffic.
2. The request passes through the rate limiting layer to verify whether the client is within the allowed request threshold. Requests exceeding the limit are rejected.
3. NexusLB checks the backend registry to identify currently available and healthy backend servers.
4. The load balancing engine applies the selected routing strategy (Round Robin, Least Connections, or IP Hash) to choose the most suitable backend.
5. The reverse proxy forwards the request to the selected backend server and waits for the response within the configured timeout limits.
6. If the backend successfully responds, the response is returned to the client. In case of failure, the reliability layer attempts a controlled retry on another healthy backend where applicable.
7. After completing the request, NexusLB updates system metrics and generates structured logs containing request details, backend information, latency, and response status.
8. The collected information is displayed through the monitoring dashboard, providing real-time visibility into system performance and backend health.

4.3 Operational Constraints

The project is designed with the following operational constraints:

- **Controlled development environment:** NexusLB will initially operate with locally hosted backend servers to allow controlled testing of routing decisions, failures, and performance behaviour.
- **Lightweight implementation:** The system will focus on understanding and implementing core distributed system concepts without depending on complex external infrastructure or managed cloud services.
- **Open-source technology stack:** The implementation will rely on freely available technologies such as Go, Git, GitHub, and Docker to maintain accessibility and reproducibility.
- **Focus on reliability and observability:** The primary objective is to demonstrate correct request handling, failure recovery, and system monitoring rather than achieving production-level performance comparable to enterprise solutions.
- **Single-instance deployment:** The initial version will run as a single NexusLB instance managing a backend pool. Distributed deployment of multiple load balancer instances will be considered as future scope.

5 Solution Approach

... engineering focus

This project follows an engineering-focused approach where the emphasis is on designing, implementing, and evaluating a reliable distributed traffic management system. The focus is on building a scalable and measurable software component rather than developing novel algorithms. The system will be developed incrementally, with each major module tested independently before integration.

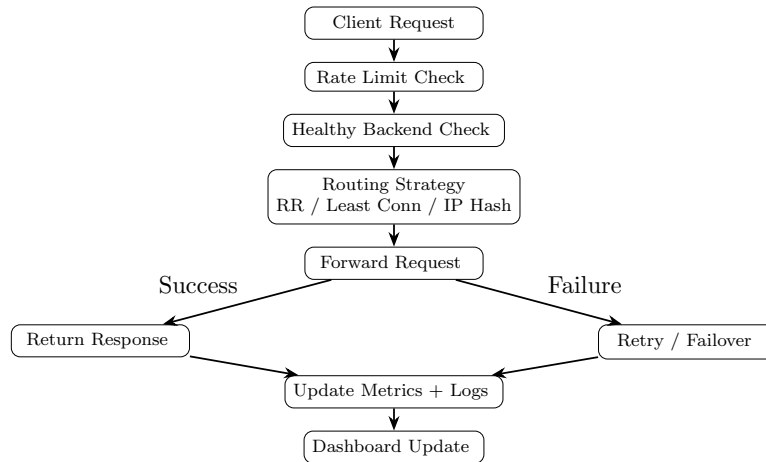


Figure 2: Request lifecycle inside NexusLB showing routing, failure handling, and monitoring updates.

5.1 Load Balancing Strategy Implementation

The routing logic will be implemented using multiple well-known load balancing strategies rather than relying on a single fixed approach.

- **Strategy-based architecture:** A common load balancing interface will be designed, allowing different routing algorithms to be implemented and replaced without affecting the rest of the system.

- **Multiple routing algorithms:** Round Robin, Least Connections, and IP Hash strategies will be implemented and evaluated based on their request distribution behaviour.
- **Configurable routing behaviour:** The active strategy will be controlled through configuration files, allowing changes without modifying the source code.

5.2 System Architecture

NexusLB will follow a modular architecture where each responsibility is separated into independent components.

- **Proxy Engine:** Handles incoming HTTP requests and forwards them to selected backend servers.
- **Backend Management:** Maintains the backend server registry and tracks server availability and connection information.
- **Reliability Layer:** Implements health checks, timeout handling, retries, and failover mechanisms to maintain service availability during failures.
- **Observability Layer:** Collects metrics, generates structured logs, and provides dashboard-based monitoring of system behaviour.

5.3 Concurrency and Request Handling

Since a load balancer must handle multiple requests simultaneously, concurrency will be a core implementation consideration.

- Incoming client connections will be handled concurrently using Go routines.
- Background processes such as health checks and metrics collection will run independently.
- Shared resources such as backend states, connection counts, and collected metrics will be protected using appropriate synchronization mechanisms to prevent race conditions.

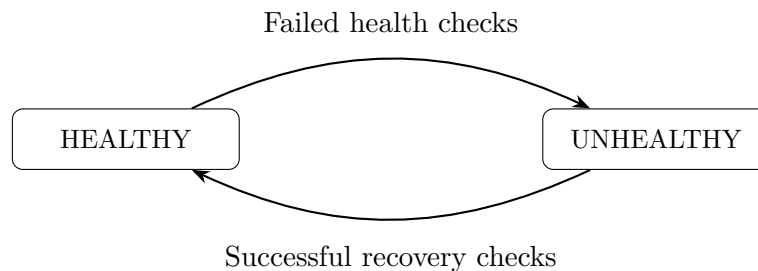


Figure 3: Backend health state transition used by NexusLB for failure detection and recovery.

5.4 Testing and Performance Evaluation

The system will be validated through controlled experiments that measure both normal operation and failure handling capabilities.

- Test individual components such as routing strategies, health state transitions, rate limiting, and retry logic independently.
- Perform integration testing using multiple simulated backend servers connected through NexusLB.

- Evaluate system behaviour during backend failures, slow responses, and sudden traffic spikes.
- Measure performance indicators such as throughput, latency, error rate, request distribution, and recovery time.

5.5 CI/CD and Incremental Delivery

Development will follow an incremental delivery approach to ensure that each stage produces a working system component.

- Maintain version control using Git and GitHub with regular commits for each implemented module.
- Use automated testing to verify system correctness after changes.
- Gradually integrate advanced features such as health monitoring, failover handling, metrics collection, and dashboard visualization after the core proxy functionality is stable.

6 Evaluation Criterion

... measurable and attributable

The success of NexusLB will be evaluated using measurable performance, reliability, and observability metrics directly related to the core system objectives. The evaluation will focus on how effectively the system distributes traffic, handles failures, and maintains visibility into backend behaviour.

6.1 Primary evaluation metric

Request handling efficiency: The primary evaluation metric will measure how effectively NexusLB manages incoming requests across multiple backend servers while maintaining acceptable latency and throughput.

- **Target:** Demonstrate efficient request distribution with measurable improvements in backend utilization compared to direct client-to-server communication.
- **Attribution:** Measured using load-testing results, including request distribution across servers, throughput, response latency, and successful request percentage.

6.2 Secondary evaluation metrics

- **Load distribution accuracy:** Measure how evenly incoming requests are distributed among backend servers under different routing strategies such as Round Robin, Least Connections, and IP Hash.
- **Failure detection and recovery time:** Evaluate how quickly NexusLB detects an unhealthy backend, removes it from routing, and restores it after recovery.
- **Latency and throughput:** Measure average response time, percentile latency (P95/P99), and requests handled per second under different traffic conditions.
- **Fault tolerance:** Evaluate system behaviour during backend crashes, slow responses, and network failures by measuring failed requests, successful retries, and service continuity.
- **Observability quality:** Assess whether collected metrics and structured logs accurately represent request flow, backend health, errors, and system performance.

6.3 Validation plan

- Deploy multiple simulated backend servers and generate controlled client traffic through NexusLB.
- Compare different routing strategies by analysing request distribution and performance characteristics.
- Perform controlled failure experiments by stopping backend servers, introducing delays, and generating traffic spikes.
- Record system metrics such as throughput, latency, error rate, recovery time, and backend health transitions to validate the effectiveness of the implementation.

7 Scalability

...with foresight

NexusLB is designed with scalability considerations at both the request-handling and system architecture levels. Although the initial implementation focuses on a single load balancer instance managing a controlled backend pool, the modular design allows future expansion.

- **Backend scalability:** The backend registry is designed to support multiple backend servers, allowing additional servers to be added or removed through configuration rather than requiring changes to the core implementation.
- **Algorithm scalability:** The strategy-based load balancing design allows new routing approaches, such as weighted load balancing or latency-aware routing, to be introduced without modifying existing components.
- **Traffic scalability:** Concurrent request handling using Go routines allows NexusLB to process multiple client connections simultaneously while maintaining shared state safely.
- **Future distributed scaling:** A future version could support multiple NexusLB instances working together. However, this introduces additional challenges such as distributed configuration management, shared rate limiting, and centralized monitoring, which are outside the scope of the current implementation.

8 Engineering Available Heuristic

The project follows established engineering practices and existing infrastructure concepts rather than attempting to develop new load balancing algorithms. The objective is to understand, implement, and evaluate proven techniques used in real-world systems.

- **Existing routing strategies:** Well-known algorithms such as Round Robin, Least Connections, and IP Hash will be implemented and compared instead of designing new balancing algorithms.
- **Incremental development:** The system will be developed in stages, starting with basic request forwarding and gradually adding routing strategies, health monitoring, reliability mechanisms, and observability features.
- **Modular engineering approach:** Components such as proxy handling, backend management, health checks, metrics collection, and logging will remain independent to simplify testing and future improvements.
- **Practical validation:** System behaviour will be evaluated through controlled experiments and measurable benchmarks rather than theoretical assumptions.

9 Project scope and deliverables

... iteration-friendly

The project scope focuses on developing a functional reverse proxy and load balancer with intelligent routing, reliability mechanisms, and system observability. Development will follow an incremental approach, where the core traffic management functionality is implemented first, followed by advanced monitoring and reliability features.

9.1 Initial deliverable

The first iteration will focus on establishing the core functionality of NexusLB:

- **Basic reverse proxy:** Implement an HTTP proxy capable of receiving client requests, forwarding them to backend servers, and returning responses.
- **Backend server management:** Create a backend registry to maintain information about available servers and their current health status.
- **Round Robin load balancing:** Implement the first routing strategy to distribute incoming requests across multiple backend servers.
- **Configurable backend setup:** Support defining backend servers and basic system settings through a configuration file.
- **Unit testing and validation:** Test core components such as request forwarding, backend selection, and configuration handling.
- **Version control and development workflow:** Maintain the project repository with structured commits and incremental feature integration.

9.2 Subsequent deliverables

Later iterations will extend the system with reliability, performance, and monitoring capabilities:

- **Advanced routing strategies:** Implement Least Connections and IP Hash algorithms and compare their performance under different workloads.
- **Health monitoring and failover:** Add periodic health checks, automatic removal of unhealthy backends, and recovery-based reintegration.
- **Reliability mechanisms:** Implement request timeouts, controlled retries, and failover handling for backend failures.
- **Traffic control features:** Introduce rate limiting to protect backend services during sudden traffic increases.
- **Metrics and dashboard:** Develop monitoring capabilities to display request rate, latency, error rate, backend health, and traffic distribution.
- **Structured logging and tracing:** Add detailed request logs containing request identifiers, backend information, response status, and processing time.
- **Benchmarking and evaluation:** Conduct experiments measuring throughput, latency, failure recovery time, and routing behaviour.

10 Risks and Mitigations

- **Concurrency and synchronization issues:** Multiple client requests, health checks, and metrics collection running simultaneously may introduce race conditions or inconsistent system states. **Mitigation:** Design shared components carefully and use synchronization mechanisms such as mutexes or atomic operations. Test concurrent behaviour early using controlled workloads.
- **Scope expansion:** Implementing a complete production-grade load balancer involves several advanced features beyond the semester timeline. **Mitigation:** Prioritize the core MVP consisting of reverse proxy functionality, routing strategies, health checks, and fault handling. Advanced features such as distributed deployment will remain future scope.
- **Performance measurement variability:** Benchmark results may vary depending on hardware limitations, system background processes, and test conditions. **Mitigation:** Keep testing conditions consistent, repeat experiments multiple times, and report measured results rather than assumed performance values.
- **Failure handling complexity:** Retry and failover mechanisms may introduce unexpected behaviour, especially for requests that are not safe to repeat. **Mitigation:** Implement controlled retries with clear limitations and evaluate failure scenarios carefully before enabling automatic recovery.
- **Dashboard development affecting core progress:** Building monitoring and visualization features may consume time needed for the main backend implementation. **Mitigation:** Develop the dashboard only after the core routing and reliability components are stable, using already available metrics APIs.
- **Security limitations:** Features such as TLS termination and request validation may not provide production-level security without extensive auditing. **Mitigation:** Clearly define the project as an educational implementation focused on infrastructure concepts rather than a production replacement for systems like NGINX or HAProxy.

11 Summary

This project proposes the development of NexusLB, a custom reverse proxy and load balancer designed to improve traffic management, reliability, and observability in distributed web applications. The system focuses on intelligent request routing, health-aware backend management, fault tolerance, and real-time monitoring through a modular engineering approach.

The project emphasizes measurable evaluation through controlled experiments involving routing performance, failure recovery, latency, throughput, and system behaviour under different workloads. By implementing established infrastructure concepts such as load balancing strategies, health checks, retries, and metrics collection, NexusLB aims to provide practical understanding of how modern distributed systems handle scalability and reliability challenges.

The project remains focused on engineering implementation and evaluation rather than developing novel algorithms. Its contribution lies in building a functional, measurable, and extensible infrastructure component that demonstrates the principles behind real-world traffic management systems.